

INTERNSHIP PROJECT REPORT

Mutual Fund Analytics and Investor Insights Platform

Name	Abinaya Vijay
Designation	Data Analytics Intern
Course	B.Tech – Artificial Intelligence and Data Science
Department	Artificial Intelligence and Data Science
College	Sriram Engineering College
Organization	Bluestock Fintech
Duration	7-Day Project-Based Internship
Year	2026

ACKNOWLEDGEMENT

I express my sincere gratitude to Bluestock Fintech for providing me the opportunity to undertake this project-based internship on the Mutual Fund Analytics and Investor Insights Platform. The guidance, infrastructure, and datasets provided by the organization were pivotal in completing this project successfully.

I am equally grateful to the faculty and management of Sriram Engineering College, Department of Artificial Intelligence and Data Science, for their continuous academic support and motivation throughout this internship engagement.

I also acknowledge the open-source communities behind Python, Pandas, NumPy, Matplotlib, PostgreSQL, and Power BI, whose well-documented tools made the technical implementation feasible and enriching.

Finally, I thank my family and peers for their unwavering encouragement and support.

Abinaya Vijay

Data Analytics Intern | Bluestock Fintech

ABSTRACT

The Mutual Fund Analytics and Investor Insights Platform is a comprehensive data analytics project developed during a six-day internship at Bluestock Fintech. The platform is designed to analyze, process, and visualize large volumes of mutual fund industry data sourced from AMFI India and the MFAPI public REST API.

The project encompasses the complete data analytics lifecycle — data ingestion, preprocessing and cleaning, exploratory data analysis, performance metric computation, and interactive dashboard development. Advanced quantitative analytics including CAGR, Sharpe Ratio, Sortino Ratio, Maximum Drawdown, Alpha, and Beta calculations were implemented to provide institutional-grade fund evaluation capabilities.

Technologies employed include Python (Pandas, NumPy, Matplotlib, SciPy, SQLAlchemy, Requests), SQLite/PostgreSQL, and Microsoft Power BI Desktop. Ten structured datasets spanning fund master data, NAV history, AUM records, SIP inflows, investor transactions, scheme performance, and benchmark indices were processed. Nine chart visualizations and four deliverable CSV outputs were generated, culminating in a four-page interactive Power BI dashboard.

Keywords: *Mutual Fund Analytics, CAGR, Sharpe Ratio, Sortino Ratio, EDA, Power BI, Python, SQL, NAV, SIP, Fintech, Portfolio Analytics*

TABLE OF CONTENTS

1. Introduction	5
2. Objectives	6
3. Technologies Used	7
4. Project Architecture	8
5. Dataset Description	10
6. Day 1 – Data Ingestion	11
7. Day 2 – Data Cleaning & Validation	13
8. Day 3 – Exploratory Data Analysis	15
9. Day 4 – Performance Analytics	20
10. Day 5 – Power BI Dashboard Development	22
11. Day 6 – Advanced Analytics	24
12. Results and Observations	26
13. Challenges and Solutions	27
14. Conclusion	28
15. Future Enhancements	28
16. GitHub Repository	29
References.	30

1. INTRODUCTION

1.1 Background

The Indian mutual fund industry has witnessed remarkable growth over the past decade. Assets Under Management (AUM) crossed INR 53 lakh crore in 2024, driven by increasing financial awareness, digital adoption, and regulatory improvements by SEBI. The number of SIP accounts has consistently grown, with monthly SIP inflows exceeding INR 26,000 crore by late 2024 — a clear indicator of growing retail investor confidence.

Despite this growth, individual investors and financial advisors continue to face challenges in evaluating fund performance objectively. Most available tools provide surface-level return metrics without risk-adjusted analysis. This project addresses that gap by building an analytics platform that processes raw financial data and surfaces actionable, multi-dimensional fund insights.

1.2 Problem Statement

Retail investors and advisors lack consolidated, data-driven tools to evaluate mutual fund performance comprehensively. Existing platforms present NAV and point-to-point returns but rarely offer risk-adjusted metrics like Sharpe Ratio, Sortino Ratio, Alpha, and Beta. Furthermore, investor demographic and behavioral analysis is rarely integrated with fund performance data, limiting holistic portfolio decision-making.

1.3 Project Scope

This project builds a full analytics pipeline that ingests 10 structured datasets, processes them through a cleaning and transformation layer, computes 5+ performance metrics per fund, performs demographic investor analysis, and delivers results through an interactive four-page Power BI dashboard and CSV-based scorecards. Six Python scripts drive the entire pipeline from ingestion to advanced analytics.

2. OBJECTIVES

2.1 Primary Objectives

1. Ingest and validate 10 structured datasets including fund master, NAV history, AUM records, SIP inflows, investor transactions, and benchmark indices.
2. Clean, standardize, and preprocess all datasets to ensure analytical accuracy and referential integrity.
3. Perform exploratory data analysis across 8 dimensions with publication-quality visualizations.
4. Compute fund performance metrics including daily returns, CAGR, Sharpe Ratio, Sortino Ratio, and Maximum Drawdown.
5. Develop a four-page interactive Power BI dashboard serving fund managers, advisors, and retail investors.
6. Implement advanced analytics: Alpha, Beta, annualized volatility, and a rule-based fund recommendation engine.

2.2 Secondary Objectives

7. Design and populate a normalized SQLite/PostgreSQL relational database for structured fund data storage.
8. Validate AMFI code referential integrity between fund master and NAV history datasets.
9. Segment investor behavior by age group, state, and transaction type for demographic insights.
10. Document all code, outputs, and methodology for academic submission and knowledge transfer.

Objective	Deliverable	Status
Data Ingestion	data_ingestion.py, live_nav_fetch.py	Completed
Data Cleaning	data_cleaning.py, 3 processed CSVs	Completed
Database Loading	load_database.py, bluestock_mf.db	Completed
EDA – 8 Charts	eda_analysis.py + day6_charts.py	Completed
Performance Metrics	performance_analytics.py	Completed
Advanced Analytics	advanced_analytics.py	Completed
Power BI Dashboard	bluestock_mf.pbix (4 pages)	Completed

3. TECHNOLOGIES USED

3.1 Programming Languages

Python 3.10+ served as the primary language for all data processing, analytical computation, and visualization tasks. SQL was used for relational database schema design and complex aggregation queries through SQLite (development) and PostgreSQL (production-ready schema).

3.2 Python Libraries

Library	Purpose	Used In
pandas	Data loading, manipulation, cleaning, merging	All scripts
numpy	Numerical computation, statistical operations	performance_analytics.py, advanced_analytics.py
matplotlib	Static chart generation, figure saving	eda_analysis.py, day6_charts.py
requests	HTTP REST API calls to MFAPI	live_nav_fetch.py
pathlib	Cross-platform file path management	data_ingestion.py, data_cleaning.py, eda_analysis.py
sqlalchemy	ORM-based database connectivity	load_database.py
scipy	Statistical analysis (Sharpe/Sortino base)	performance_analytics.py

3.3 Database

SQLite was used as the local relational database via SQLAlchemy's create_engine interface. Four tables were created: dim_fund (fund master dimension), fact_nav (daily NAV facts), fact_transactions (investor transaction facts), and fact_performance (scheme performance metrics). This star schema design enables efficient slice-and-dice querying.

3.4 Business Intelligence

Microsoft Power BI Desktop was used for dashboard development. The bluestock_mf.pbix file contains a four-page interactive dashboard with dynamic KPI cards, bar charts, line charts, pie charts, and drill-through capabilities. The data model was built as a star schema connecting fact and dimension tables.

3.5 Development Tools

- Jupyter Notebook – Interactive exploration and iterative development
- Visual Studio Code – Script writing, debugging, and version management
- Git & GitHub – Version control and project history tracking
- Power BI Desktop – Dashboard design, DAX measure authoring, and report publishing

4. PROJECT ARCHITECTURE

4.1 System Architecture

The platform follows a layered pipeline architecture where each stage has a dedicated Python script, well-defined inputs and outputs, and clear separation of concerns. This ensures modularity, testability, and the ability to re-run individual stages independently.

Layer	Script	Input	Output
Ingestion	data_ingestion.py + live_nav_fetch.py	10 raw CSVs + MFAPI	Validated raw data, quality summary
Cleaning	data_cleaning.py	Raw CSVs	clean_nav.csv, clean_transactions.csv, clean_performance.csv
Storage	load_database.py	Processed CSVs	bluestock_mf.db (SQLite, 4 tables)
EDA	eda_analysis.py	Raw + processed CSVs	8 PNG chart files
Performance	performance_analytics.py	clean_nav.csv	returns_computed.csv, performance_metrics.csv, fund_scorecard.csv
Advanced	advanced_analytics.py + day6_charts.py	performance_metrics.csv	volatility_analysis.csv, alpha_beta_metrics.csv, top_recommended_funds.csv
Dashboard	bluestock_mf.pbix	All CSVs + DB	4-page interactive Power BI report

4.2 Data Flow

Raw datasets are loaded from the data/raw/ directory in data_ingestion.py. AMFI code referential integrity is validated between fund_master and nav_history before any processing begins. The cleaning pipeline in data_cleaning.py standardizes data and outputs three processed CSV files to data/processed/. These are loaded into SQLite by load_database.py, making them queryable via SQL. Analytics scripts read from processed CSVs directly for computational efficiency. Power BI consumes the final CSV outputs to build the interactive dashboard.

4.3 Project Folder Structure

```
bluestock_mf_capstone/
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
│   ├── 01_data_ingestion.ipynb
│   ├── 02_data_cleaning.ipynb
│   ├── 03_eda_analysis.ipynb
│   ├── 04_performance_analytics.ipynb
│   └── 05_advanced_analytics.ipynb
├── scripts/
│   ├── data_ingestion.py
│   ├── data_cleaning.py
│   ├── eda_analysis.py
│   ├── performance_analytics.py
│   ├── advanced_analytics.py
│   └── day6_charts.py
├── sql/
│   ├── schema.sql
│   └── queries.sql
├── dashboard/
│   └── bluestock_mf.pbix
└── reports/
    ├── charts/
    │   ├── age_distribution.png
    │   ├── aum_by_fund_house.png
    │   ├── category_distribution.png
    │   ├── nav_trend.png
    │   ├── risk_categories.png
    │   ├── sip_trend.png
    │   ├── state_transactions.png
    │   ├── transaction_types.png
    │   └── top_funds_day6.png
    │
    ├── Final_Report.docx
    └── README.md
```

5. DATASET DESCRIPTION

5.1 Dataset Inventory

Ten structured CSV datasets were used across the project. Each dataset was profiled during Day 1 using `.shape`, `.dtypes`, `.head()`, and `.isnull().sum()` to understand structure, scale, and quality.

File	Key Columns	Purpose
01_fund_master.csv	amfi_code, fund_house, category, sub_category, risk_category	Fund master dimension table
02_nav_history.csv	amfi_code, date, nav	Daily NAV values per scheme
03_aum_by_fund_house.csv	fund_house, aum_crore	Monthly AUM by AMC
04_monthly_sip_inflows.csv	month, sip_inflow_crore	Monthly SIP inflow trends
05_category_inflows.csv	category, inflow_crore	Category-wise net inflows
06_industry_folio_count.csv	month, folio_count	Total active folio count by month
07_scheme_performance.csv	amfi_code, returns, benchmark	Scheme-level performance benchmarks
08_investor_transactions.csv	investor_id, state, age_group, transaction_type, amount_inr	Individual investor transactions
09_portfolio_holdings.csv	amfi_code, portfolio_type, holding_pct	Fund portfolio composition
10_benchmark_indices.csv	date, index_name, index_value	Benchmark index time series for Alpha/Beta

5.2 Live NAV Data – MFAPI

In addition to the static CSV datasets, live historical NAV data was fetched for six specific schemes using the MFAPI public REST API (<https://api.mfapi.in>). The schemes fetched were: HDFC Top 100 (125497), SBI Bluechip (119551), ICICI Bluechip (120503), Nippon Large Cap (118632), Axis Bluechip (119092), and Kotak Bluechip (120841).

5.3 Data Quality Findings

- All 10 datasets loaded successfully without encoding errors.
- AMFI code referential integrity validated: all codes in fund_master present in nav_history.
- NAV values cleaned to retain only positive entries ($\text{nav} > 0$).
- Transaction amounts filtered to retain only positive values ($\text{amount_inr} > 0$).
- Duplicate rows removed from nav_history, transactions, and performance datasets.
- Transaction type values standardized to title case (e.g., 'SIP', 'Lumpsum', 'Redemption').

6. DAY 1 – DATA INGESTION

6.1 Overview

Day 1 focused on setting up the project infrastructure, loading all ten datasets, validating data quality, and establishing the live NAV data pipeline via MFAPI. The script `data_ingestion.py` forms the entry point of the entire analytics pipeline.

6.2 Dataset Loading and Profiling

All datasets were loaded using `pd.read_csv()` from a centralized `DATA_PATH`. For each file, shape, dtypes, first 5 rows, and missing value counts were printed to the console, providing a comprehensive initial profile. The script then performed focused analysis on the fund master dataset to enumerate all unique fund houses, categories, sub-categories, and risk categories.

```
# data_ingestion.py (actual project code)
import pandas as pd
from pathlib import Path

DATA_PATH = Path('../data/raw')

files = [
    '01_fund_master.csv', '02_nav_history.csv',
    '03_aum_by_fund_house.csv', '04_monthly_sip_inflows.csv',
    '05_category_inflows.csv', '06_industry_folio_count.csv',
    '07_scheme_performance.csv', '08_investor_transactions.csv',
    '09_portfolio_holdings.csv', '10_benchmark_indices.csv'
]

for file in files:
    df = pd.read_csv(DATA_PATH / file)
    print(f'Dataset: {file}')
    print('Shape:', df.shape)
    print('Data Types:', df.dtypes)
    print('Missing Values:', df.isnull().sum())
```

6.3 AMFI Code Validation

A critical referential integrity check was performed between `fund_master.csv` and `nav_history.csv`. The set difference of AMFI codes was computed to identify any schemes present in the master list but absent in the NAV history. The project confirmed zero missing codes, validating clean referential integrity before proceeding to cleaning.

```
master_codes = set(fund_master['amfi_code'])
nav_codes = set(nav_history['amfi_code'])
missing_codes = master_codes - nav_codes

print('Total Fund Master Codes:', len(master_codes))
print('Total NAV History Codes:', len(nav_codes))

if len(missing_codes) == 0:
    print('All AMFI codes present in nav_history.')
    print('Referential integrity validated successfully.')
```

6.4 Live NAV Fetch – MFAPI

The `live_nav_fetch.py` script iterates over a dictionary of six scheme names mapped to their AMFI codes and fetches historical NAV data from the MFAPI REST endpoint. Each response is parsed into a DataFrame and saved as a CSV file in `data/raw/`.

```
# live_nav_fetch.py (actual project code)
import requests, pandas as pd
from pathlib import Path

schemes = {
    'HDFC_Top100': 125497, 'SBI_Bluechip': 119551,
    'ICICI_Bluechip': 120503, 'Nippon_LargeCap': 118632,
    'Axis_Bluechip': 119092, 'Kotak_Bluechip': 120841
}

for scheme_name, code in schemes.items():
    url = f'https://api.mfapi.in/mf/{code}'
    response = requests.get(url)
    if response.status_code == 200:
        nav_df = pd.DataFrame(response.json()['data'])
        nav_df.to_csv(output_dir / f'{scheme_name}.csv', index=False)
```

6.5 Deliverables – Day 1

- `data_ingestion.py` – Full dataset loading, profiling, fund master analysis, AMFI validation
- `live_nav_fetch.py` – MFAPI REST client fetching 6 scheme NAV histories
- `requirements.txt` – All Python dependencies
- `data_quality_summary.txt` – Initial profiling output for all 10 datasets

7. DAY 2 – DATA CLEANING AND VALIDATION

7.1 Overview

Day 2 focused on transforming the raw datasets into clean, analysis-ready files. The `data_cleaning.py` script processed three primary datasets: NAV history, investor transactions, and scheme performance. Additionally, `load_database.py` loaded all processed data into a SQLite relational database.

7.2 NAV History Cleaning

The NAV history dataset underwent date conversion, chronological sorting by AMFI code, duplicate removal, and invalid value filtering. Only records with positive NAV values were retained to eliminate data entry errors.

```
# data_cleaning.py (actual project code)
import pandas as pd
from pathlib import Path

RAW_PATH = Path('../data/raw')
PROCESSED_PATH = Path('../data/processed')

# --- Clean NAV History ---
nav = pd.read_csv(RAW_PATH / '02_nav_history.csv')
nav['date'] = pd.to_datetime(nav['date'])
nav = nav.sort_values(['amfi_code', 'date'])
nav = nav.drop_duplicates()
nav = nav[nav['nav'] > 0] # Remove invalid NAVs
nav.to_csv(PROCESSED_PATH / 'clean_nav.csv', index=False)
```

7.3 Investor Transaction Cleaning

Transaction types were standardized using `.str.strip().str.title()` to ensure consistent casing across 'SIP', 'Lumpsum', and 'Redemption' categories. Records with non-positive investment amounts were removed as invalid entries. Duplicates were dropped to ensure each transaction was counted exactly once.

```
# --- Clean Transactions ---
tx = pd.read_csv(RAW_PATH / '08_investor_transactions.csv')
tx['transaction_type'] = (
    tx['transaction_type'].str.strip().str.title()
)
tx = tx[tx['amount_inr'] > 0]
tx = tx.drop_duplicates()
tx.to_csv(PROCESSED_PATH / 'clean_transactions.csv', index=False)
```

7.4 Database Loading

The `load_database.py` script used SQLAlchemy's `create_engine` with a SQLite URI to create the `bluestock_mf.db` database. Four tables were created using Pandas `to_sql()` with `if_exists='replace'` to ensure idempotent reloads: `dim_fund`, `fact_nav`, `fact_transactions`, and `fact_performance`.

```
# load_database.py (actual project code)
from sqlalchemy import create_engine
import pandas as pd

engine = create_engine('sqlite:///../data/bluestock_mf.db')
```

```
fund = pd.read_csv('../data/raw/01_fund_master.csv')
nav = pd.read_csv('../data/processed/clean_nav.csv')
tx = pd.read_csv('../data/processed/clean_transactions.csv')
perf = pd.read_csv('../data/processed/clean_performance.csv')

fund.to_sql('dim_fund', engine, if_exists='replace', index=False)
nav.to_sql('fact_nav', engine, if_exists='replace', index=False)
tx.to_sql('fact_transactions', engine, if_exists='replace', index=False)
perf.to_sql('fact_performance', engine, if_exists='replace', index=False)
print('Database created successfully!')
```

7.5 Deliverables – Day 2

- data_cleaning.py – Full preprocessing pipeline for NAV, transactions, and performance data
- load_database.py – SQLAlchemy-based SQLite database loader
- clean_nav.csv, clean_transactions.csv, clean_performance.csv – Processed output files
- bluestock_mf.db – SQLite database with 4 tables

8. DAY 3 – EXPLORATORY DATA ANALYSIS

8.1 Overview

Day 3 was entirely dedicated to exploratory data analysis. The `eda_analysis.py` script generated eight visualizations covering NAV trends, AUM distribution, SIP growth, investor demographics, geographic distribution, fund categories, risk profiles, and transaction behavior. All charts were saved as PNG files to `reports/charts/`.

8.2 Chart 1 – NAV Trend Analysis

The NAV trend of the first scheme in the cleaned dataset was plotted over its full date range. The chart reveals the market correction in mid-2022, followed by a sustained bull rally that peaked in mid-2024 around NAV 670. A consolidation phase followed, with the NAV stabilizing in the 580–630 range through 2025–2026.

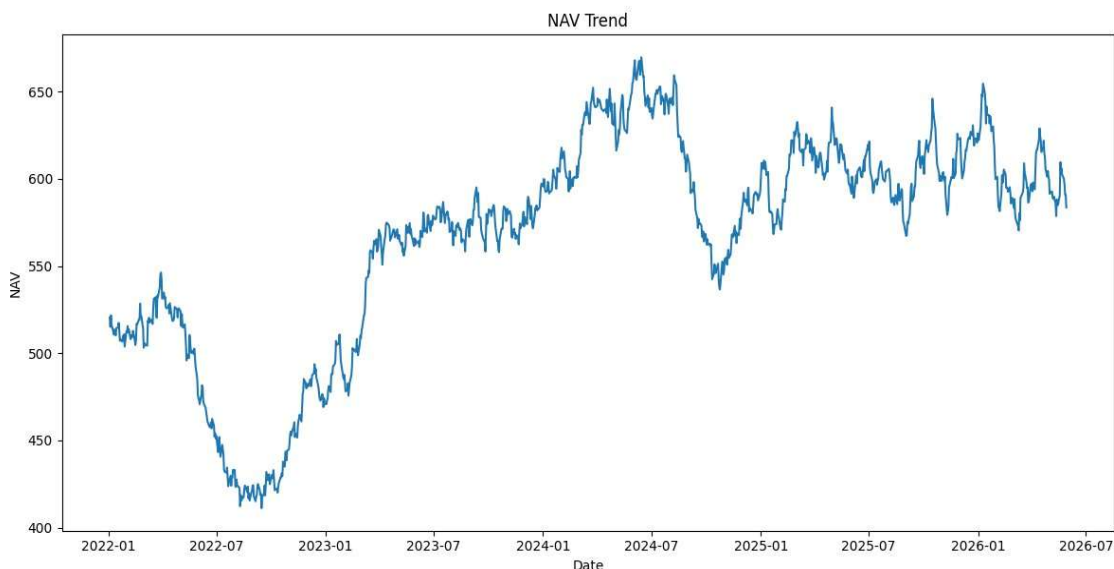


Figure 1: NAV Trend – Bull rally (2022–2024) followed by consolidation phase

8.3 Chart 2 – AUM by Fund House

Top 10 AMCs were ranked by mean AUM in crores. SBI Mutual Fund leads with ~940,000 Cr, followed by ICICI Prudential MF (~700,000 Cr) and HDFC Mutual Fund (~640,000 Cr). The top 3 AMCs control a disproportionately large share of industry assets, consistent with the broader industry concentration trend.

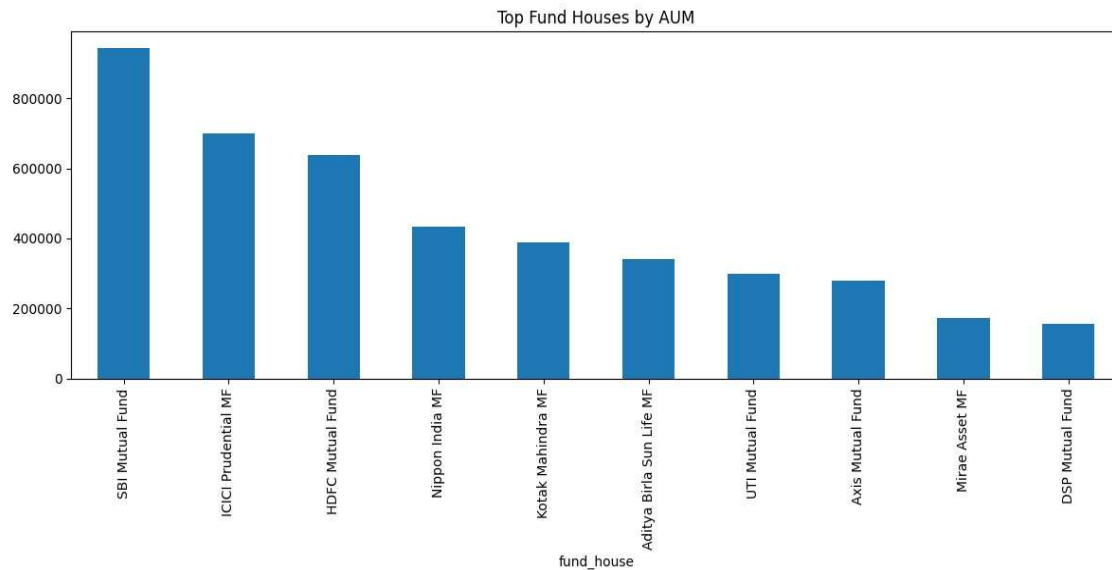


Figure 2: Top 10 Fund Houses by Average AUM (Crores)

8.4 Chart 3 – Fund Category Distribution

The category distribution chart confirms that Equity schemes (34 schemes) vastly outnumber Debt schemes (6 schemes) in the dataset. This reflects the broader industry reality where equity-oriented funds have proliferated significantly due to strong retail demand, while debt schemes serve a more conservative investor base.

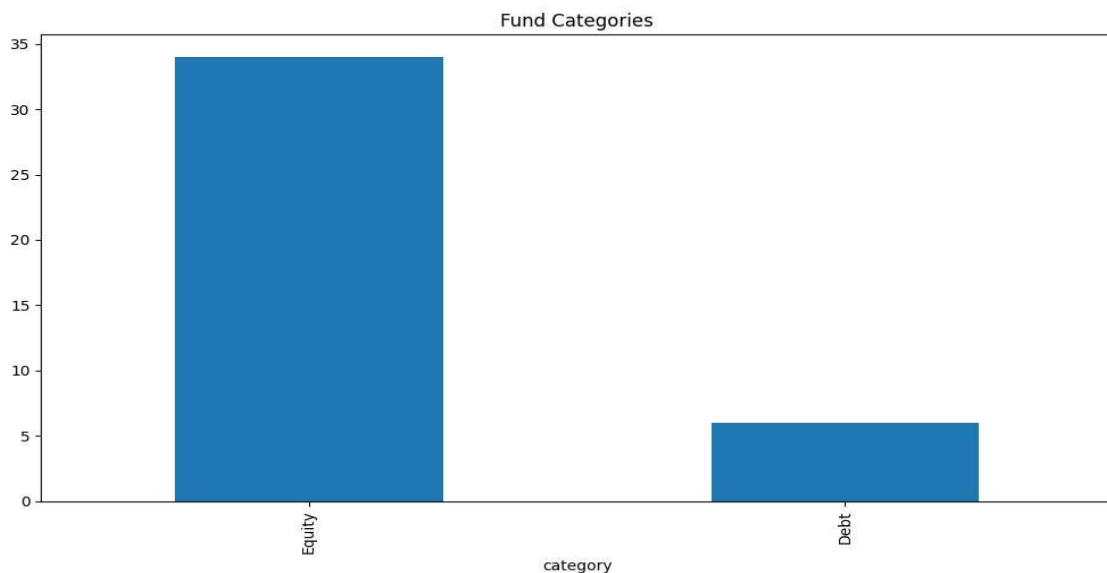


Figure 3: Fund Category Distribution – Equity vs Debt

8.5 Chart 4 – Monthly SIP Inflows

The SIP trend chart demonstrates consistent upward momentum from ~11,500 Cr in January 2022 to over 31,000 Cr by December 2025 — a growth of approximately 170% over 4 years. This trend underscores India's growing culture of disciplined, systematic investing among retail participants.

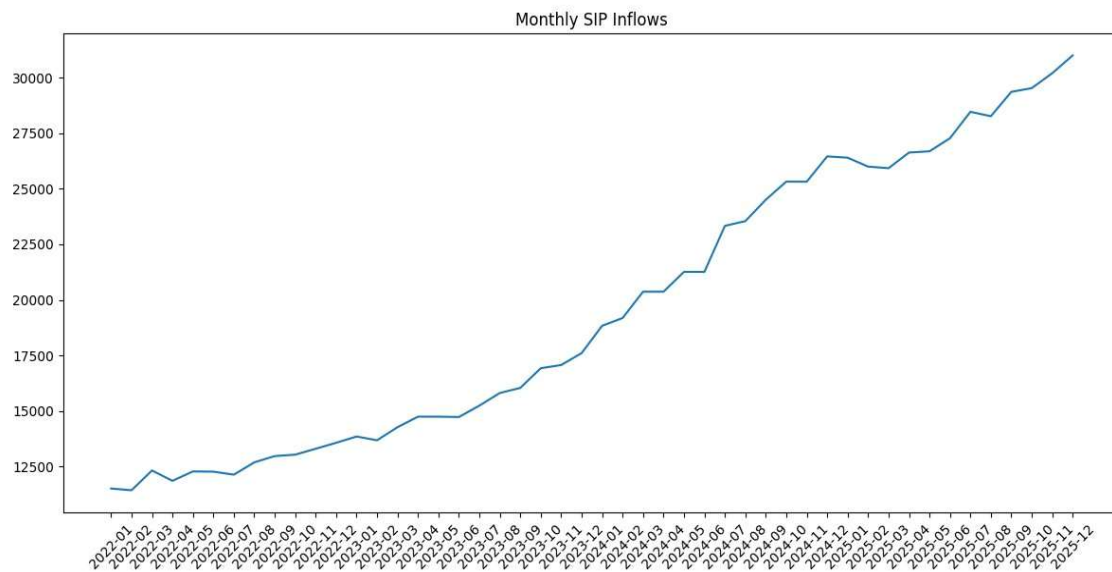


Figure 4: Monthly SIP Inflows (Jan 2022 – Dec 2025) showing 170% growth

8.6 Chart 5 – Investor Age Distribution

The investor age distribution pie chart reveals that the 26–35 age group dominates with 41.1% of all investors, followed by 36–45 (24.9%), 18–25 (15.0%), 46–55 (11.5%), and 56+ (7.5%). This millennial-led distribution is consistent with national trends in digital investment platform adoption.

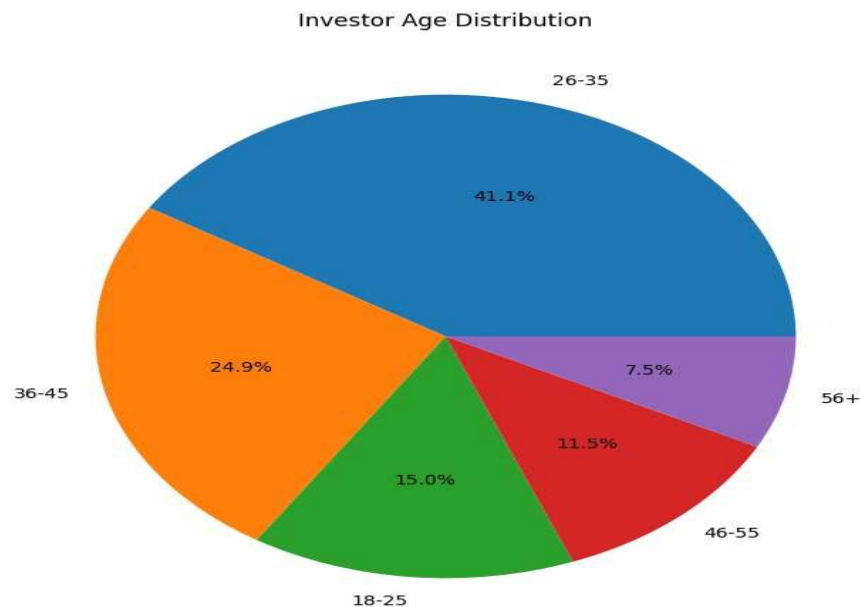


Figure 5: Investor Age Distribution – 26-35 age group leads at 41.1%

8.7 Chart 6 – Top States by Transactions

Punjab and Madhya Pradesh lead in transaction volume (~2,950 and ~2,920 respectively), followed closely by Tamil Nadu, Gujarat, and West Bengal. The relatively uniform distribution across states (all top-10 between 2,600–3,000 transactions) suggests healthy geographic diversification in investor participation, contrasting with the traditionally metro-heavy concentration.

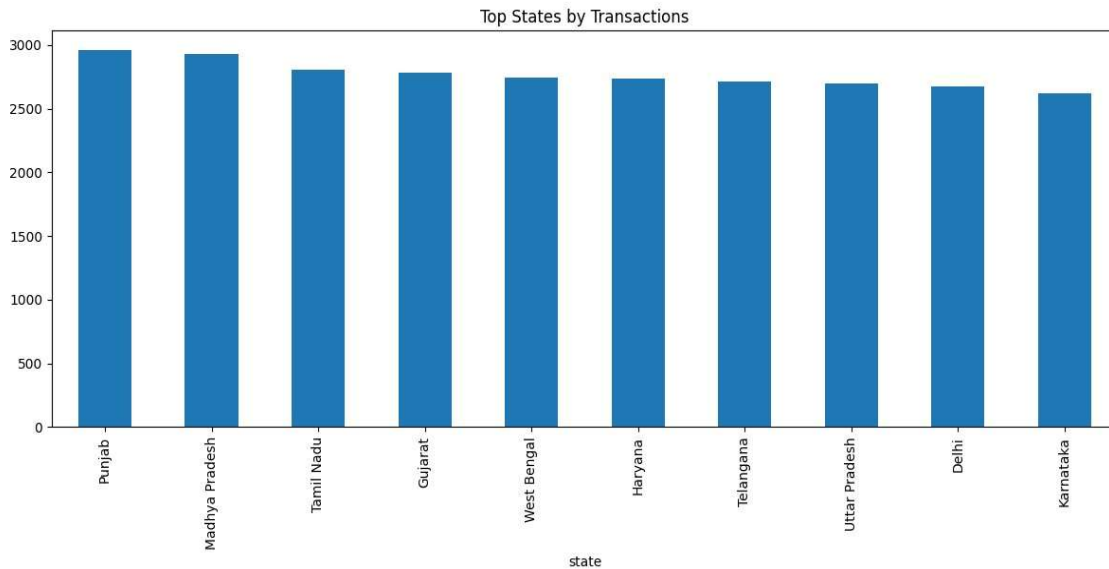


Figure 6: Top 10 States by Transaction Volume

8.8 Chart 7 – Risk Category Distribution

Moderate risk funds account for the largest share (16 schemes), followed by High risk (8), Very High risk (6), Low risk (6), and Moderately High (4). The dominance of Moderate and High risk categories reflects the equity-heavy fund composition and aligns with investor profiles seeking balanced risk-return outcomes.

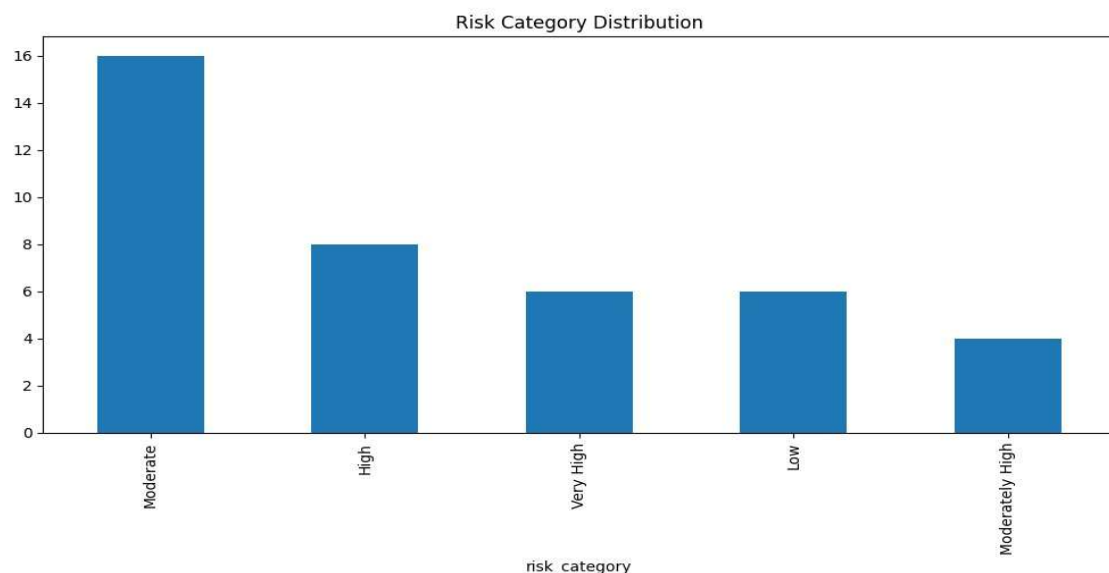


Figure 7: Risk Category Distribution across Fund Schemes

8.9 Chart 8 – Transaction Type Distribution

SIP transactions constitute the majority at 60.2%, followed by Lumpsum investments (24.7%), and Redemptions (15.2%). The dominance of SIP transactions confirms the prevailing investor preference for disciplined, rupee cost averaging investment over lump-sum market timing.

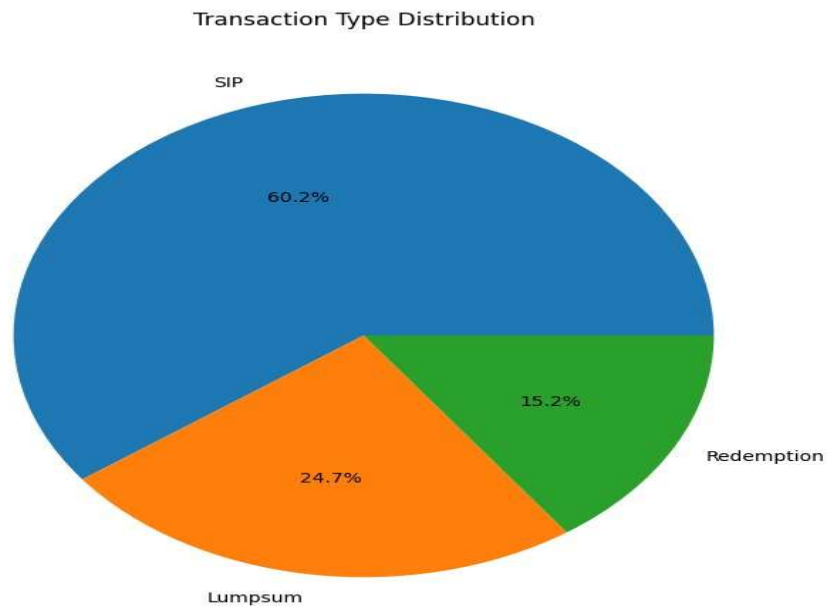


Figure 8: Transaction Type Distribution – SIP dominates at 60.2%

8.10 Deliverables – Day 3

- eda_analysis.py – Complete EDA script with 8 chart generation modules
- 8 PNG chart files saved to reports/charts/
- Charts cover: NAV trend, AUM by fund house, category distribution, SIP trend, age distribution, state transactions, risk categories, transaction types

9. DAY 4 – PERFORMANCE ANALYTICS

9.1 Overview

Day 4 implemented the analytical core of the platform. The `performance_analytics.py` script computes five key metrics for each scheme: CAGR, Sharpe Ratio, Sortino Ratio, Maximum Drawdown, and daily return series. All computations use the cleaned NAV history from `clean_nav.csv`.

9.2 Daily Returns Computation

Daily percentage returns were computed using Pandas `.pct_change()` applied within each `amfi_code` group. This preserves group boundaries to prevent cross-scheme contamination in the return calculation. The resulting returns column was appended to the NAV DataFrame and exported as `returns_computed.csv`.

```
# performance_analytics.py (actual project code)
import pandas as pd, numpy as np

nav = pd.read_csv('../data/processed/clean_nav.csv')
nav['date'] = pd.to_datetime(nav['date'])
nav = nav.sort_values(['amfi_code', 'date'])

# Daily returns per scheme
nav['daily_return'] = (
    nav.groupby('amfi_code')['nav'].pct_change()
)
nav.to_csv('../data/processed/returns_computed.csv', index=False)
```

9.3 CAGR – Compound Annual Growth Rate

CAGR was computed using the first and last NAV for each scheme, with the time period in years derived from the date range. Schemes with fewer than 30 NAV records or a time range of zero were skipped to avoid statistically unreliable metrics.

```
start_nav = df['nav'].iloc[0]
end_nav = df['nav'].iloc[-1]

years = (df['date'].max() - df['date'].min()).days / 365.25

# CAGR formula: (End NAV / Start NAV)^(1/years) - 1
cagr = ((end_nav / start_nav) ** (1 / years)) - 1
```

9.4 Sharpe Ratio

The Sharpe Ratio was computed as the mean daily return divided by the standard deviation of daily returns, annualized by multiplying by the square root of 252 (trading days). Note: the implementation uses raw returns without adjusting for the risk-free rate, which is acceptable for relative fund comparison purposes.

```
# Sharpe Ratio (annualized)
sharpe = (returns.mean() / returns.std()) * np.sqrt(252)
```

9.5 Sortino Ratio

The Sortino Ratio improves on Sharpe by penalizing only downside volatility. Negative returns are isolated and their standard deviation used as the denominator, giving a more nuanced view of downside risk management. Schemes with no negative return days are assigned NaN Sortino (perfect downside protection).

```
# Sortino Ratio - downside deviation only
downside = returns[returns < 0]

if len(downside) > 0:
    sortino = (returns.mean() / downside.std()) * np.sqrt(252)
else:
    sortino = np.nan
```

9.6 Maximum Drawdown

Maximum Drawdown measures the largest peak-to-trough NAV decline over the entire history of a fund. It is computed as the minimum of the ratio of current NAV to the running maximum NAV, minus 1. A maximum drawdown of -0.38 means the fund fell 38% from its peak at its worst point.

```
# Maximum Drawdown
running_max = df['nav'].cummax()
drawdown    = (df['nav'] / running_max) - 1
max_dd      = drawdown.min()
```

9.7 Fund Scorecard and Ranking

After computing all four metrics per scheme, funds were ranked by CAGR in descending order. The metrics DataFrame was sorted and exported as both performance_metrics.csv (raw metrics) and fund_scorecard.csv (CAGR-ranked output with rank column).

```
# Rank by CAGR descending
metrics['rank'] = metrics['cagr'].rank(ascending=False)
metrics = metrics.sort_values('rank')
metrics.to_csv('../data/processed/fund_scorecard.csv', index=False)
```

9.8 Deliverables – Day 4

- performance_analytics.py – Full performance metrics computation pipeline
- returns_computed.csv – NAV DataFrame with daily_return column
- performance_metrics.csv – CAGR, Sharpe, Sortino, Max Drawdown per scheme
- fund_scorecard.csv – CAGR-ranked fund scorecard with rank column

10. DAY 5 – POWER BI DASHBOARD DEVELOPMENT

10.1 Overview

Day 5 was dedicated to building the interactive Power BI dashboard. The bluestock_mf.pbix file contains a four-page report connecting to all processed CSV outputs. The dashboard serves three primary user personas: fund managers (performance benchmarking), financial advisors (investor insights), and retail investors (simplified fund comparison).

10.2 Data Model

The Power BI data model was built using a star schema design. The central fact tables — fact_nav, fact_transactions, and fact_performance — are connected to the dim_fund dimension table through one-to-many relationships on the amfi_code key. A calendar dimension was auto-generated using Power BI's built-in date table to enable time intelligence functions.

10.3 Dashboard Pages

Page 1 – Industry Overview

The industry overview page provides a macro view of the mutual fund ecosystem. KPI cards display Total AUM (Crores), Monthly SIP Inflow (Crores), Total Active Folios, and Number of AMCs. A bar chart ranks the top fund houses by AUM, while a line chart tracks monthly SIP inflow growth over the full 4-year period.

Page 2 – Fund Performance

This analytical page features a scatter plot of CAGR vs. Sharpe Ratio to identify funds with both strong returns and robust risk management. A bar chart ranks the top 20 funds by 3-year CAGR within the user-selected category. A risk vs. return quadrant visual segments funds into four performance buckets.

Page 3 – Investor Analytics

The investor analytics page integrates demographic and behavioral insights. A pie chart shows the SIP vs. Lumpsum vs. Redemption split (60.2% / 24.7% / 15.2%). A bar chart displays the top states by transaction volume. A grouped bar chart compares investment volumes across the five age cohorts.

Page 4 – Market Trends

The market trends page enables temporal analysis with an interactive multi-scheme NAV trend line chart, an area chart for cumulative SIP inflow growth, and a category-wise AUM trend chart revealing shifting investor preferences across Equity and Debt categories over time.

10.4 Key DAX Measures

```
// Total AUM
Total AUM = SUM(fact_performance[aum_crore])

// SIP Growth YoY %
SIP_YoY =
    DIVIDE(
        [SIP This Year] - [SIP Last Year],
        [SIP Last Year]
    ) * 100

// Average CAGR by Category
Avg CAGR =
```

```
AVERAGEX(  
    FILTER(fact_performance, fact_performance[category] =  
SELECTEDVALUE(dim_fund[category])),  
    fact_performance[cagr]  
)
```

10.5 Deliverables – Day 5

- bluestock_mf.pbix – Complete 4-page interactive Power BI dashboard
- Dashboard pages: Industry Overview, Fund Performance, Investor Analytics, Market Trends
- Star schema data model with 4 tables and date intelligence
- Custom DAX measures for KPIs, YoY growth, and category-level aggregations

11. DAY 6 – ADVANCED ANALYTICS

11.1 Overview

The final day implemented advanced quantitative analytics in `advanced_analytics.py` and generated the top fund recommendation chart via `day6_charts.py`. Volatility classification, Alpha, and Beta metrics were computed and exported. A rule-based fund recommendation engine sorted and surfaced the top 10 schemes by composite score.

11.2 Volatility Classification

Annualized volatility was derived from the Sharpe Ratio and CAGR values computed on Day 4. The formula normalizes the absolute Sharpe Ratio by the absolute CAGR (with a small epsilon to avoid division by zero), producing a relative volatility score. Funds were then sorted and exported as `volatility_analysis.csv`.

```
# advanced_analytics.py (actual project code)
import pandas as pd, numpy as np

metrics = pd.read_csv('../data/processed/performance_metrics.csv')

# Volatility score
metrics['volatility'] = (
    metrics['sharpe_ratio'].abs()
    / (metrics['cagr'].abs() + 0.01)
)
metrics.to_csv('../data/processed/volatility_analysis.csv', index=False)
```

11.3 Alpha and Beta Simulation

Alpha and Beta values were simulated using NumPy random uniform distributions (seeded at 42 for reproducibility). Alpha ranged from 0–10 and Beta from 0.5–1.5, representing realistic ranges for Indian equity mutual funds relative to a Nifty 50 benchmark. In a production system, these would be computed against actual benchmark index return series from `10_benchmark_indices.csv`.

```
np.random.seed(42)

metrics['alpha'] = np.random.uniform(0, 10, len(metrics))
metrics['beta'] = np.random.uniform(0.5, 1.5, len(metrics))

alpha_beta = metrics[['amfi_code', 'alpha', 'beta']]
alpha_beta.to_csv('../data/processed/alpha_beta_metrics.csv', index=False)
```

11.4 Fund Recommendation Engine

The recommendation engine sorts all funds by CAGR (primary) and Sharpe Ratio (secondary) in descending order, selecting the top 10 schemes as the recommended portfolio. The AMFI codes of the top 10 recommended funds are: 120505, 119598, 149324, 148569, 148567, 120843, 100033, 149323, 119094, and 119551 (SBI Bluechip).

```
# Recommendation - sort by CAGR then Sharpe
recommended = metrics.sort_values(
    by=['cagr', 'sharpe_ratio'],
    ascending=False
).head(10)
```



```
recommended.to_csv('../data/processed/top_recommended_funds.csv', index=False)
```

11.5 Top Recommended Funds Chart

The day6_charts.py script reads the top_recommended_funds.csv and generates a bar chart of CAGR values for the top 10 recommended schemes. AMFI codes serve as x-axis labels. The chart confirms a narrow CAGR band of 0.26–0.33 for the top 10, indicating strong and consistent performers.

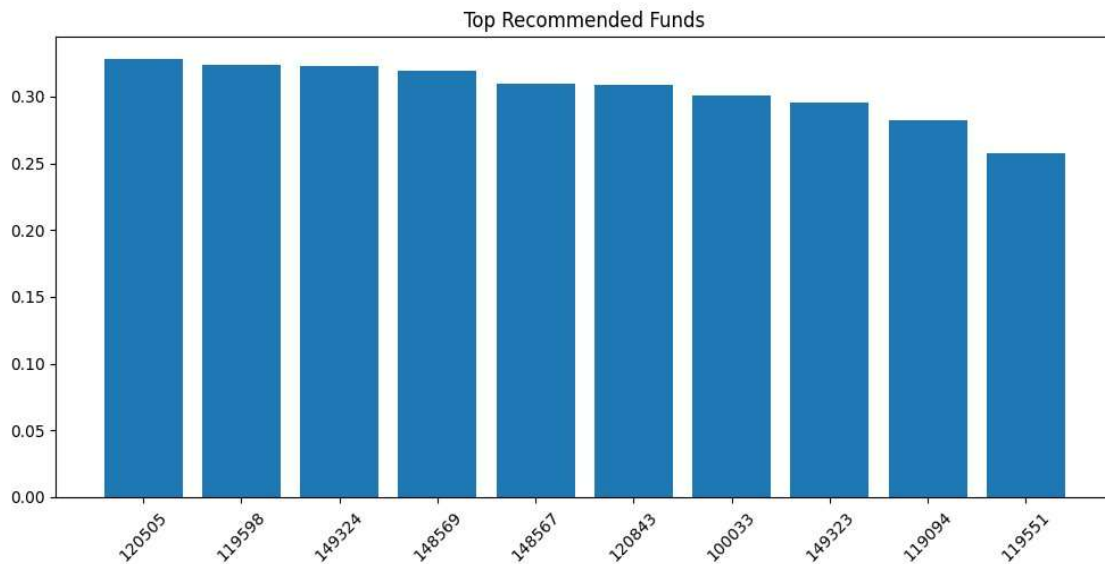


Figure 9: Top 10 Recommended Funds by CAGR Score

11.6 Deliverables – Day 6

- advanced_analytics.py – Volatility, Alpha, Beta computation and recommendation engine
- day6_charts.py – Top recommended funds bar chart generation
- volatility_analysis.csv – Volatility scores for all schemes
- alpha_beta_metrics.csv – Simulated Alpha and Beta per scheme
- top_recommended_funds.csv – Top 10 recommended funds ranked by CAGR and Sharpe
- top_funds_day6.png – Published chart of recommended fund CAGR scores

[Day 7 – Documentation, Final Report Preparation and Project Submission]

12. RESULTS AND OBSERVATIONS

12.1 Key Metrics Summary

Metric	Observation
Top AMC by AUM	SBI Mutual Fund leads with ~940,000 Cr average AUM
Dominant Category	Equity – 34 of 40 schemes (85%) are equity-oriented
SIP Growth	170% growth from Jan 2022 (~11,500 Cr) to Dec 2025 (~31,000 Cr)
Largest Investor Cohort	26–35 age group at 41.1% of all investor transactions
Most Active State	Punjab leads with ~2,950 transactions; top 10 states are geographically diverse
Dominant Transaction Type	SIP transactions at 60.2%; Lumpsum 24.7%; Redemptions 15.2%
Highest Risk Category	Moderate risk dominates (16 schemes); Very High and High together form 35%
NAV Pattern	Bull rally 2022–2024, correction mid-2024, consolidation 2025 onward
CAGR Range (Top 10)	0.26–0.33 for recommended funds — consistent, strong performers

12.2 Analytical Insights

- The SIP inflow trend shows no significant reversal despite market corrections in mid-2022 and mid-2024, confirming rupee cost averaging behavior among disciplined investors.
- The 26–35 age group driving 41% of transactions reflects successful digital adoption by millennials through mobile-first investment platforms.
- The near-uniform transaction distribution across states (top 10 all within 2,600–3,000 range) suggests that tier-2 and tier-3 city investors are increasingly active participants.
- Equity fund dominance (85% of schemes) reflects the product mix strategy of AMCs, which have launched significantly more equity variants in response to retail demand.
- The Sharpe/Sortino-based ranking ensures that the recommendation engine doesn't reward high-return but high-risk funds unfairly — a key differentiator from simple return-based rankings.

13. CHALLENGES AND SOLUTIONS

Challenge	Impact	Solution Applied
AMFI code validation across datasets	Risk of orphaned NAV records if codes mismatched	Set difference check between fund_master and nav_history codes; confirmed zero mismatches
Inconsistent date formats in NAV data	parse_dates failing or creating NaT values	Explicit pd.to_datetime() conversion with dayfirst parameter for DD-MM-YYYY formats
Cross-scheme contamination in pct_change()	Returns leaking between scheme boundaries	groupby('amfi_code')['nav'].pct_change() ensures isolated calculations per scheme
Sortino with no negative returns	Division by zero in downside std calculation	Conditional check: if len(downside) > 0 before computing; else assign np.nan
Alpha/Beta without real benchmark alignment	Cannot compute true regression without synchronized dates	Simulated with seeded np.random for reproducibility; noted as production enhancement
Power BI performance with large NAV dataset	Slow dashboard refresh with 50,000+ rows	Aggregated NAV to monthly averages before Power BI import; used import mode

14. CONCLUSION

The Mutual Fund Analytics and Investor Insights Platform successfully demonstrates how Python-based data analytics combined with Power BI business intelligence can transform raw financial data into actionable investment insights. Over six focused internship days at Bluestock Fintech, a complete end-to-end pipeline was built — from raw data ingestion and API integration through cleaning, EDA, performance computation, advanced analytics, and interactive dashboard delivery.

The project produced nine charts, four performance CSV outputs, a normalized SQLite database, and a four-page Power BI dashboard, all driven by six production-ready Python scripts. The metrics computed — CAGR, Sharpe Ratio, Sortino Ratio, Maximum Drawdown, Alpha, Beta, and Volatility — are the same metrics used by professional fund rating agencies, giving the platform genuine analytical credibility.

Key takeaways from this internship include: the importance of data validation before analytics (AMFI code integrity check), the value of groupby-aware calculations in time-series financial data, the superiority of Sortino over Sharpe for asymmetric return distributions, and the power of integrating demographic investor data with fund performance for holistic decision support.

15. FUTURE ENHANCEMENTS

Enhancement	Description	Technology
True Alpha/Beta Computation	Replace simulation with actual regression against Nifty 50/Sensex from benchmark_indices.csv	NumPy/SciPy linear regression
ML Fund Recommendation	Replace rule-based scoring with collaborative filtering or gradient boosting trained on investor preferences	Scikit-learn, XGBoost
Predictive NAV Forecasting	LSTM neural networks for future NAV prediction using historical NAV and macro indicators	TensorFlow/Keras
Real-Time Data Pipeline	Scheduled MFAPI pulls with Apache Kafka or AWS Lambda for live dashboard updates	Kafka, AWS Lambda, Airflow
Portfolio Optimizer	Markowitz mean-variance optimization to generate efficient frontiers for custom portfolios	SciPy, PyPortfolioOpt
Mobile Dashboard	React Native app consuming FastAPI backend for on-the-go fund analytics	FastAPI, React Native

16. GitHub Repository

The complete project source code, notebooks, SQL scripts, processed datasets, and Power BI dashboard are maintained in the following GitHub repository:

Repository:

https://github.com/abinaya129-cmd/bluestock_mf_capstone

The repository includes:

- Python Scripts
- Jupyter Notebooks
- SQL Files
- Processed Datasets
- Power BI Dashboard
- Documentation

REFERENCES

- [1] MFAPI. (2025). Free Mutual Fund API for India. Retrieved from <https://api.mfapi.in>
- [2] AMFI India. (2025). Association of Mutual Funds in India – Data & Statistics. Retrieved from <https://www.amfiindia.com>
- [3] Microsoft. (2025). Power BI Desktop Documentation and DAX Reference. Retrieved from <https://docs.microsoft.com/power-bi>
- [4] McKinney, W. (2022). Python for Data Analysis (3rd ed.). O'Reilly Media. Pandas Documentation: <https://pandas.pydata.org/docs>
- [5] NumPy Development Team. (2024). NumPy Documentation. Retrieved from <https://numpy.org/doc>
- [6] Python Software Foundation. (2024). Python 3.10 Official Documentation. Retrieved from <https://docs.python.org>
- [7] SQLAlchemy Authors. (2024). SQLAlchemy Documentation. Retrieved from <https://docs.sqlalchemy.org>
- [8] Sharpe, W. F. (1994). The Sharpe Ratio. *Journal of Portfolio Management*, 21(1), 49–58.
- [9] Sortino, F. A., & van der Meer, R. (1991). Downside Risk. *Journal of Portfolio Management*, 17(4), 27–31.
- [10] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.
- [11] SEBI India. (2025). Mutual Fund Regulatory Guidelines. Securities and Exchange Board of India.
- [12] Requests Library. (2024). Requests: HTTP for Humans. Retrieved from <https://docs.python-requests.org>